

Maestría en Software

Autora: Ing. Patsy Malena Prieto MSc.



| POSGRADOS |

Bienvenida

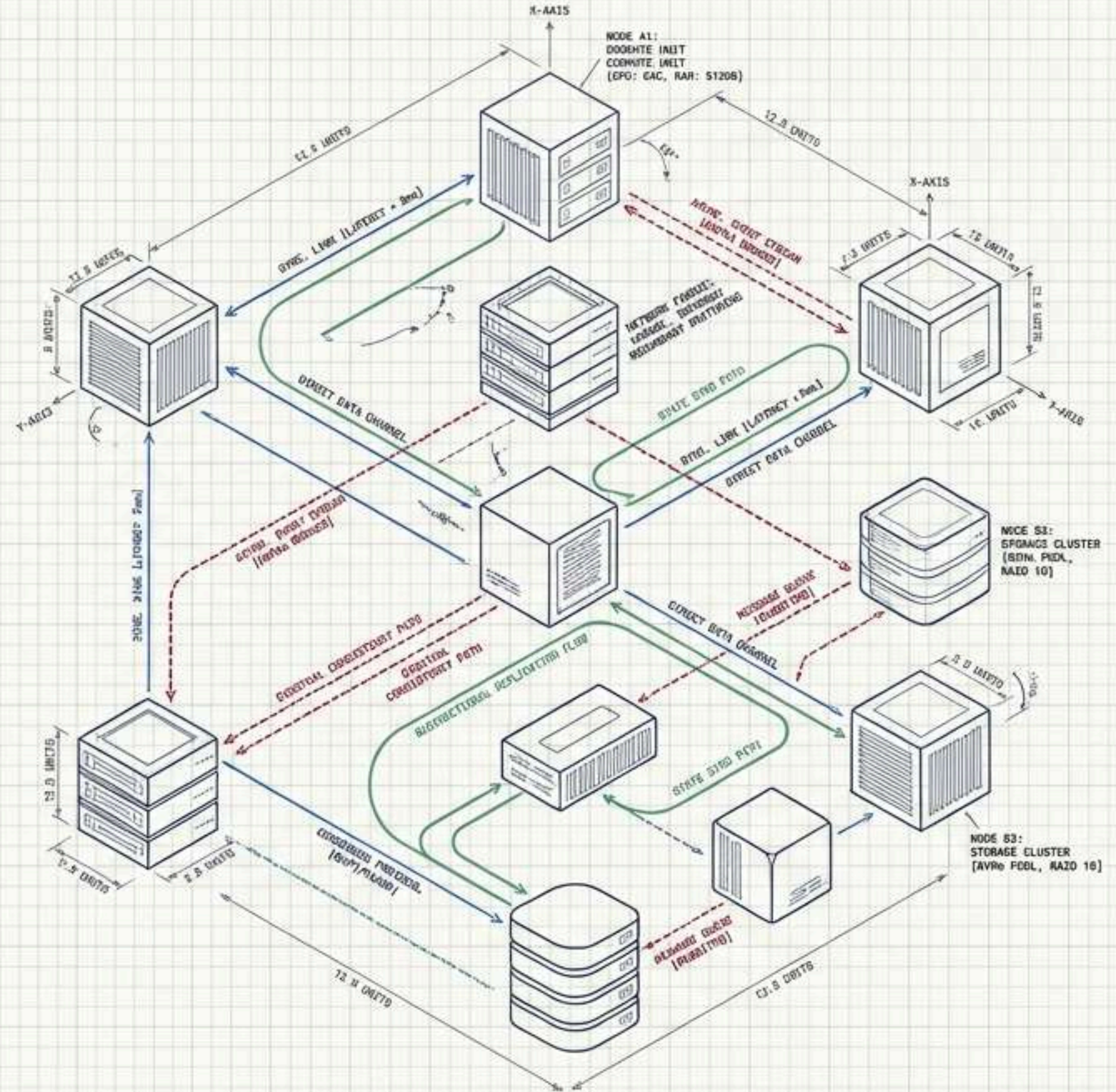
UNIDAD 2 -

Tema: Arquitectura y Patrones para sistemas dirigidos por APIs

- Arquitecturas de software orientados a APIs
- Patrones de diseño de software orientados a APIs

Estilos Arquitectónicos Modernos: El Playbook de Decisiones

Navegando trade-offs, escalabilidad y latencia en sistemas distribuidos bajo el estándar ISO/IEC 25010.

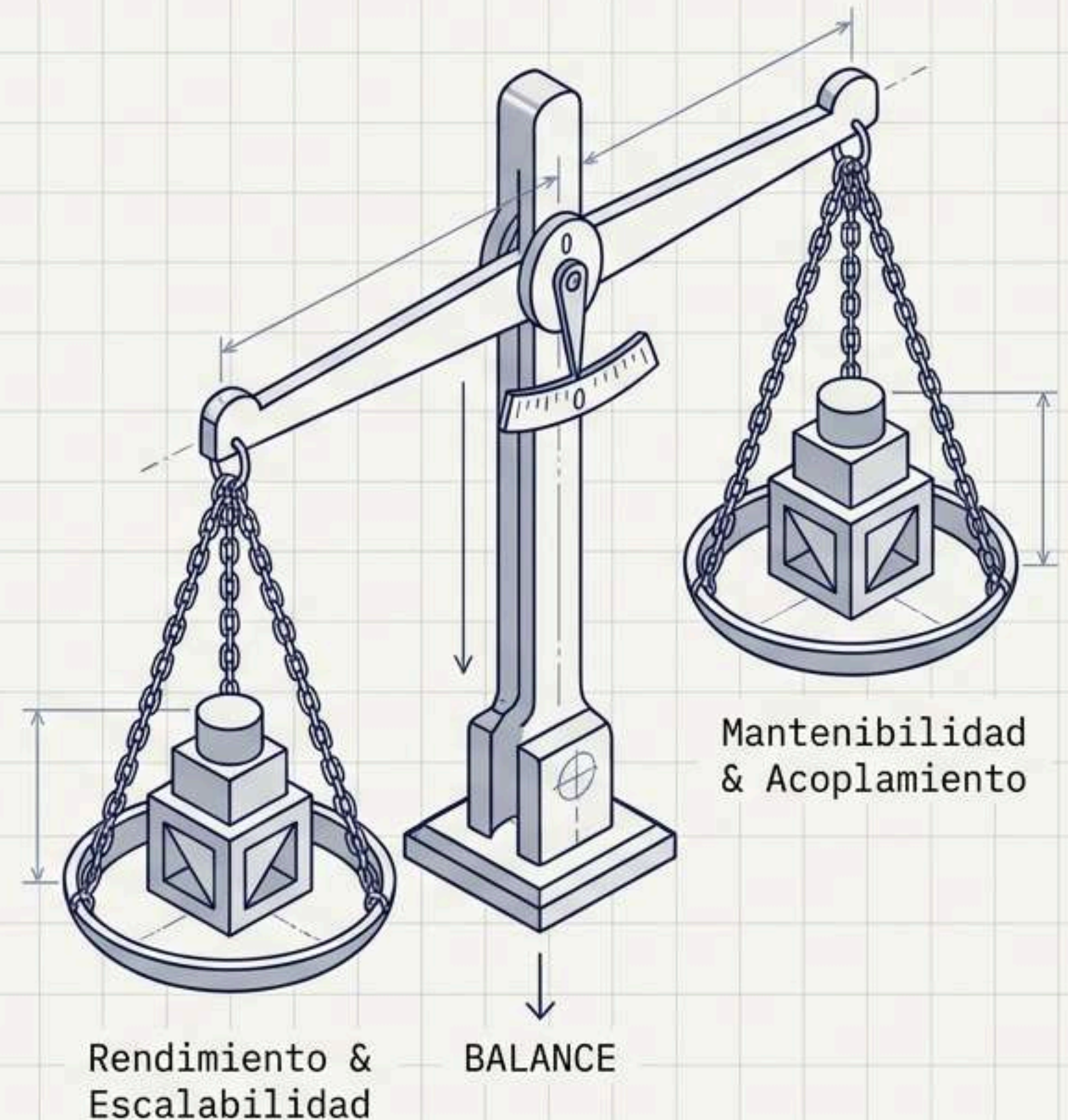


El Diseño Arquitectónico es el Arte del Trade-off

No existe la bala de plata: Toda decisión tecnológica implica sacrificar un atributo de calidad en favor de otro. El contexto dicta la elección.

ISO 25010 como brújula analítica: Evaluaremos cada paradigma basándonos empíricamente en su latencia, nivel de acoplamiento, seguridad y consumo de infraestructura.

El Ecosistema Híbrido: Sistemas complejos modernos (como plataformas de e-commerce o IoT) no eligen una arquitectura. Orquestan múltiples estilos simultáneamente según el dominio específico del problema.



Modelo de Calidad del Software: ISO/IEC 25010

El modelo de calidad ISO/IEC 25010 clasifica las propiedades del software en ocho características principales. Cada una se desglosa en subcaracterísticas específicas que permiten evaluar el rendimiento, la seguridad y la usabilidad de un producto digital.

Operación y Experiencia de Usuario

Idoneidad Funcional

Grado en que el producto proporciona funciones que satisfacen necesidades declaradas e implícitas.



Eficiencia de Rendimiento

Relación entre el nivel de rendimiento y la cantidad de recursos utilizados.



Usabilidad

Facilidad de aprendizaje, operabilidad, protección ante errores y accesibilidad para el usuario.



Fiabilidad

Capacidad del sistema para mantener su nivel de rendimiento bajo condiciones establecidas.

- Madurez
- Disponibilidad
- Tolerancia a fallos
- Recuperabilidad



Seguridad y Evolución Técnica

Seguridad

Protección de información y datos para que personas no autorizadas no puedan acceder a ellos.



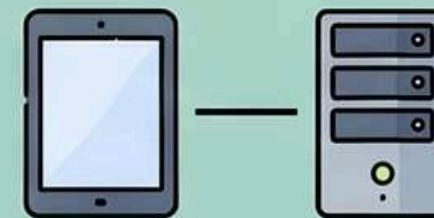
Mantenibilidad

Facilidad con la que el software puede ser modificado, corregido o mejorado.



Compatibilidad

Capacidad de dos o más sistemas para intercambiar información o realizar sus funciones compartiendo hardware.



Portabilidad

Facilidad con la que el software puede ser transferido de un entorno a otro.





REST: El Estándar Universal de Integración

[HTTP/1.1]

[JSON]

[OpenAPI / Swagger]

Filosofía

Diseño orientado a recursos identificados por URIs uniformes y manipulados mediante verbos HTTP estandarizados. Su naturaleza agnóstica lo convierte en el estándar de facto para APIs públicas.

Caso Práctico NeoMarket

Integración de API con sistemas POS de Tiendas Partner. Garantiza compatibilidad universal y aprovecha la madurez del ecosistema de herramientas.

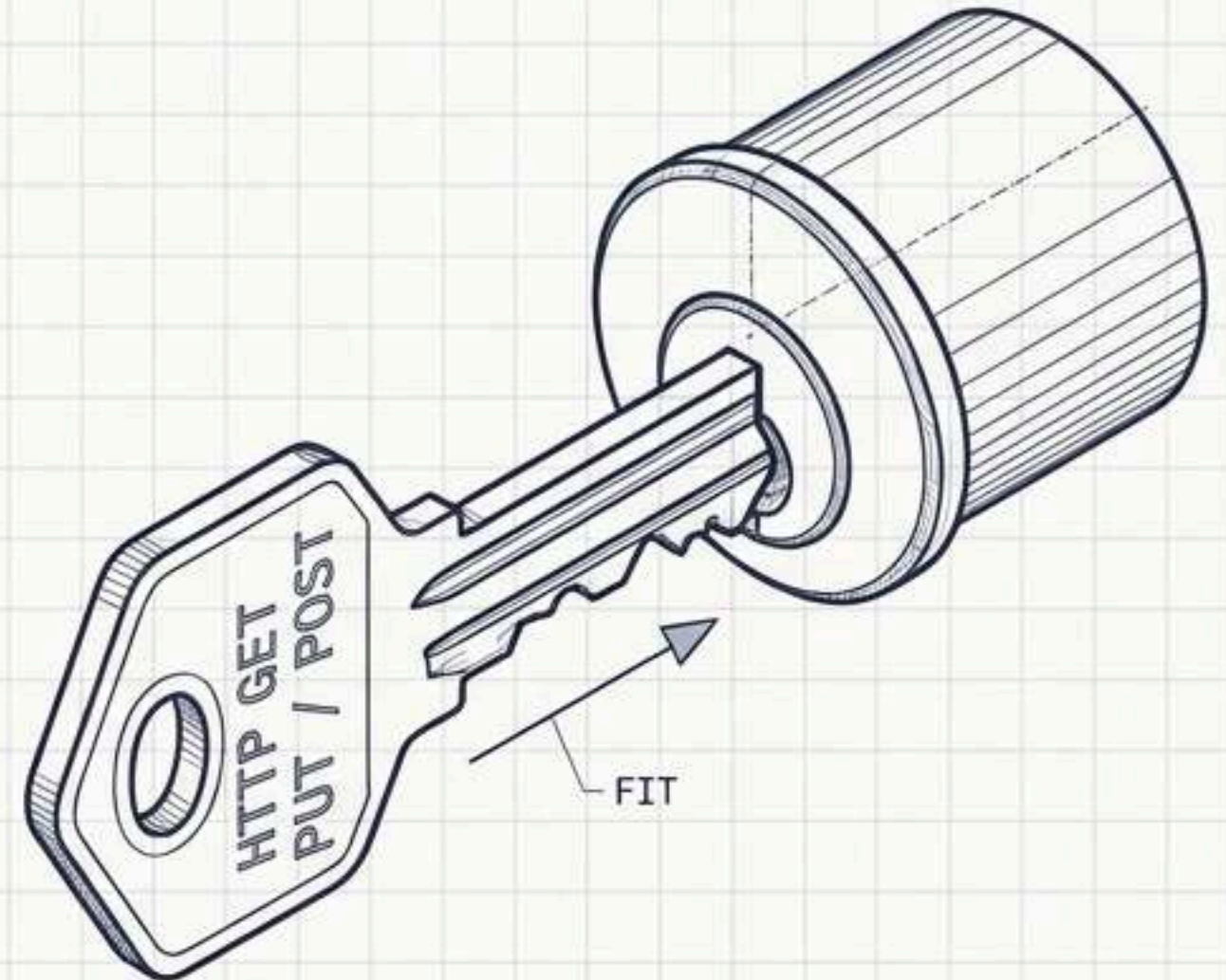
Trade-off Clave

Ventaja ↗

Excelente facilidad de cacheo distribuido (nativo a nivel HTTP vía cabeceras Cache-Control, Varnish, CDNs).

Costo ↘

Latencia media debido a payloads JSON pesados (texto plano) y posible over-fetching.





GraphQL: El Controlador a Medida para Interfaces

[POST /graphql] [SDL - Schema Definition Language]

Filosofía

Arquitectura orientada a consultas donde el cliente dicta la estructura exacta de la respuesta mediante un endpoint único. Erradica el over-fetching (datos sobrantes) y under-fetching (múltiples llamadas).

Caso Práctico NeoMarket

Backend-For-Frontend (BFF) Móvil. Permite renderizar pantallas complejas de catálogos anidados con una sola petición de red.

Trade-off Clave

A

Ventaja

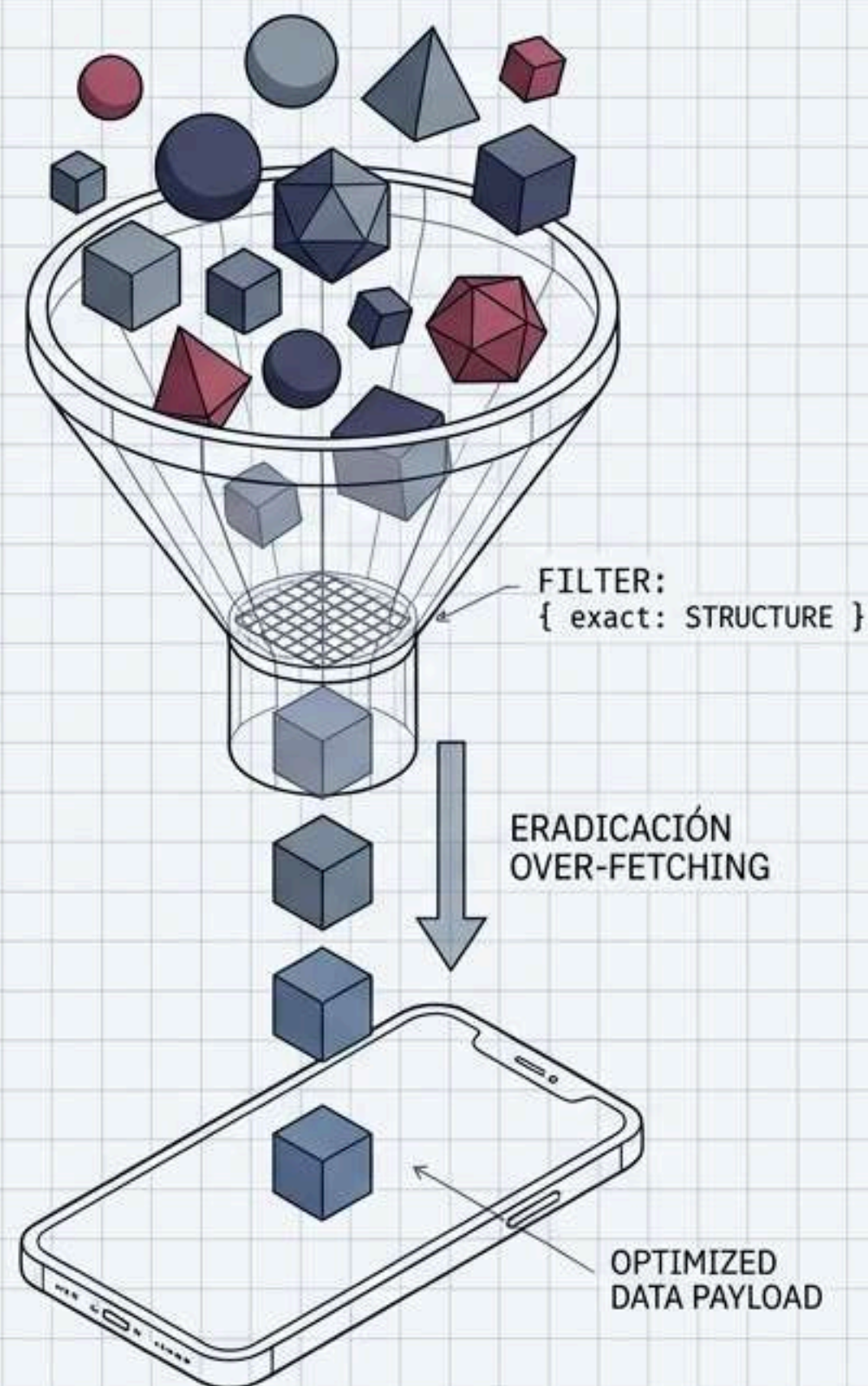
Optimización drástica de la red y autonomía total para los equipos de Frontend.



B

Costo

Transfiere la carga computacional al servidor (sobrecarga de CPU por parseo de consultas dinámicas). Se pierde el cacheo nativo de red al usar siempre HTTP POST.





gRPC: El Motor Interno de Ultra-Baja Latencia

[HTTP/2]

[Protobuf]

[Binary]

Filosofía

Framework RPC de alto rendimiento de Google.
Abandona el texto plano (JSON/XML) en favor de Protocol Buffers (Protobuf) para la serialización binaria compacta ejecutada nativamente sobre HTTP/2.

Caso Práctico NeoMarket

Tráfico East-West. Comunicación síncrona interna para chequear el inventario en milisegundos durante el proceso de pago crítico.

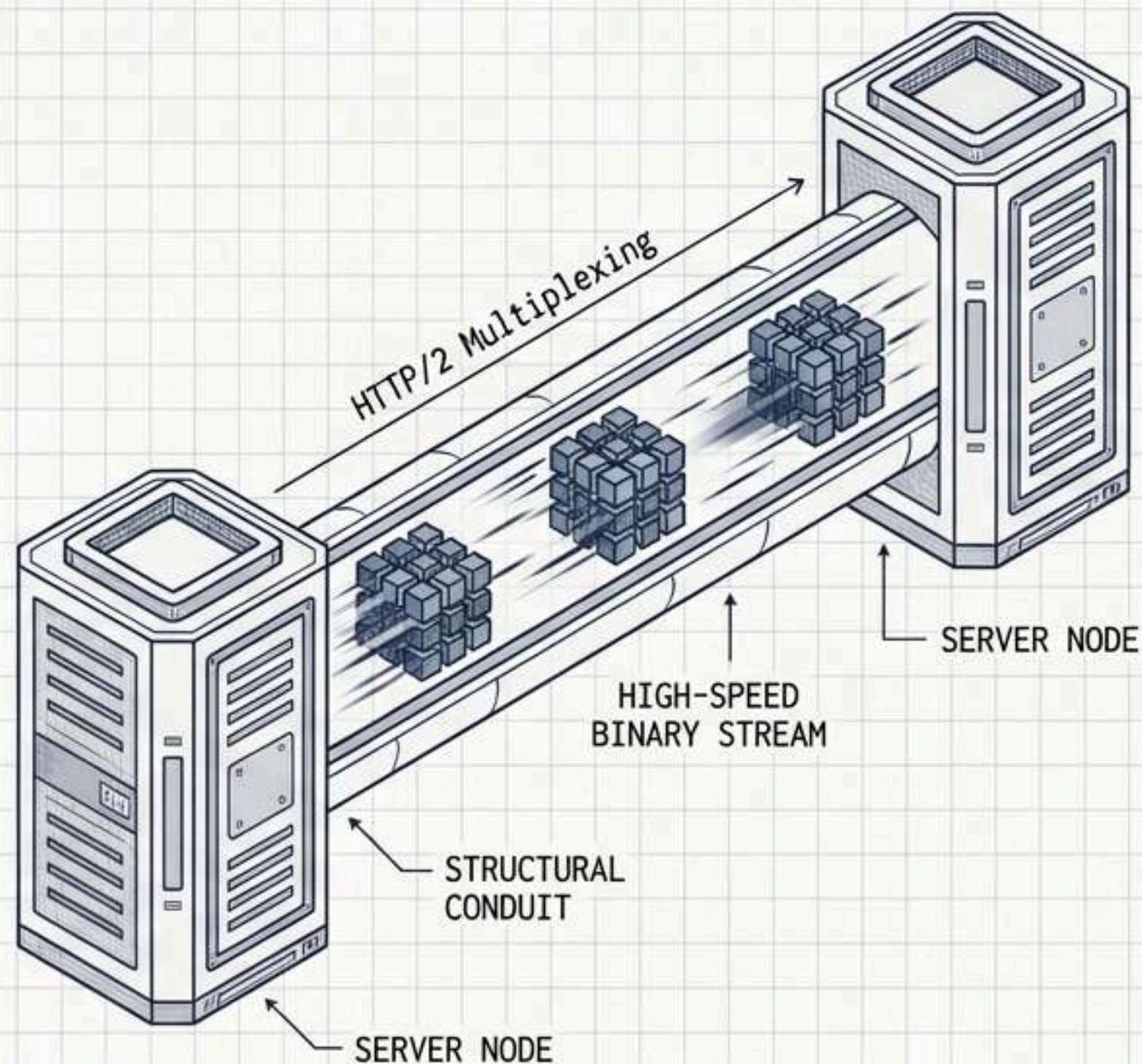
Trade-off Clave

↑ Ventaja

Latencia ultra-baja y rendimiento excepcional para arquitecturas internas de alta densidad.

↓ Costo

Alto acoplamiento contractual. Requiere la generación, compilación y distribución estricta de stubs (SDKs) entre cliente y servidor.





EDA (Event-Driven): Desacoplamiento Total en Tiempo y Espacio

[Asíncrono] [Kafka / RabbitMQ] [AsyncAPI]

Arquitectura asíncrona basada en la producción, detección y consumo de eventos de estado a través de un broker intermediario. Los productores emiten datos sin saber quién los consumirá, ni cuándo.

Caso Práctico NeoMarket

Procesamiento de compras durante Ofertas Flash. Los eventos se encolan, evitando la saturación y caída de las bases de datos transaccionales.

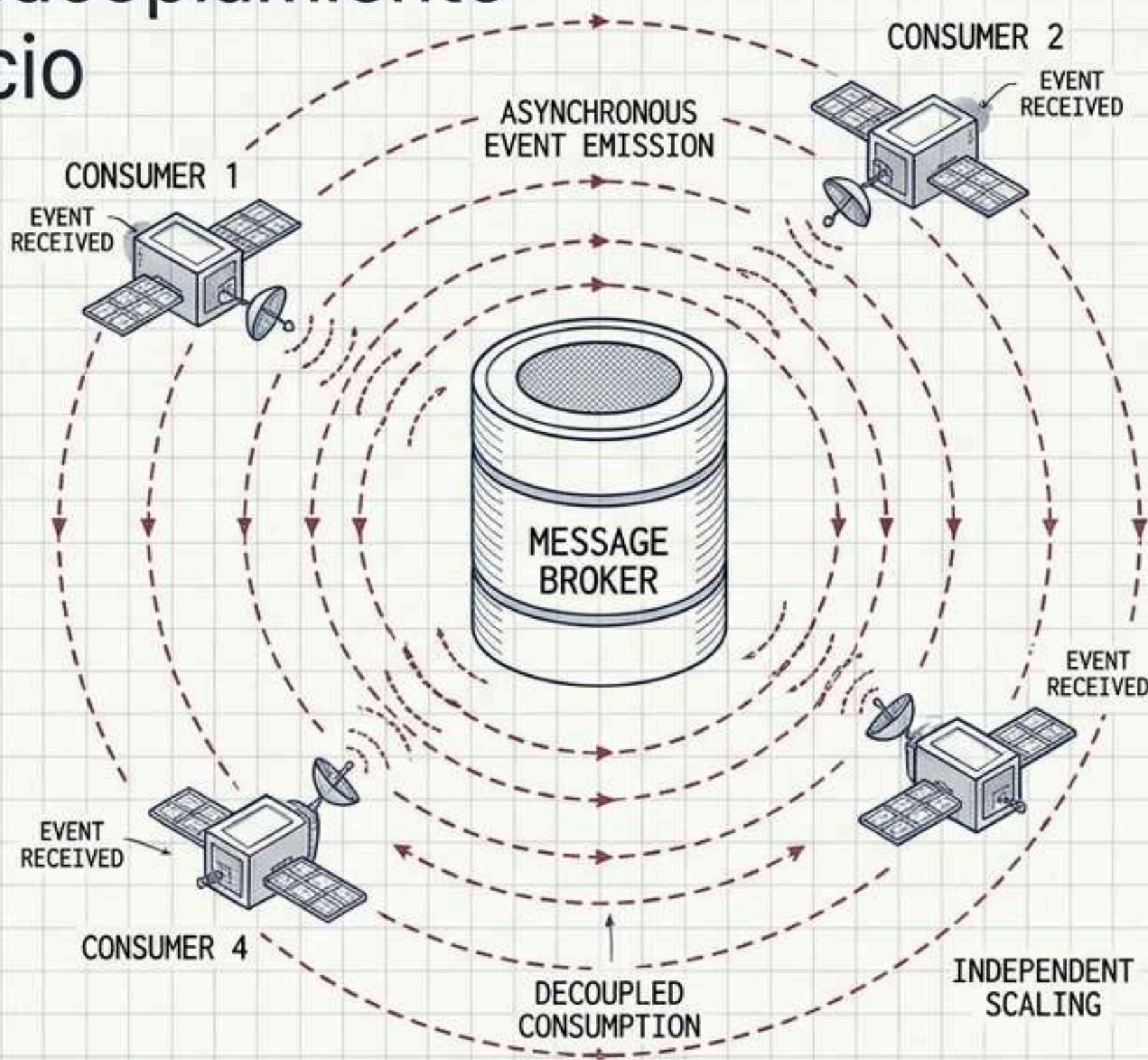
Trade-off Clave

↑ Ventaja

Escalabilidad extrema con capacidad nativa para absorber picos masivos de tráfico imprevisto.

↓ Costo

Depuración altamente compleja. Exige implementar Distributed Tracing para rastrear flujos y alta probabilidad de comportamientos emergentes.





WebSockets: El Flujo Continuo Bidireccional

[TCP] [Full-Duplex] [Upgrade: websocket]

Filosofía

Protocolo TCP permanente que inicia con una actualización HTTP. Establece un canal único, Stateful (con estado) y abierto en ambas direcciones. Cero overhead de reconexión para envíos continuos.

Caso Práctico VitalsGuard

Aplicación IoT que recibe un flujo en vivo (stream) de latidos por minuto de pacientes mayores, graficando curvas en tiempo real sin requerir polling constante.

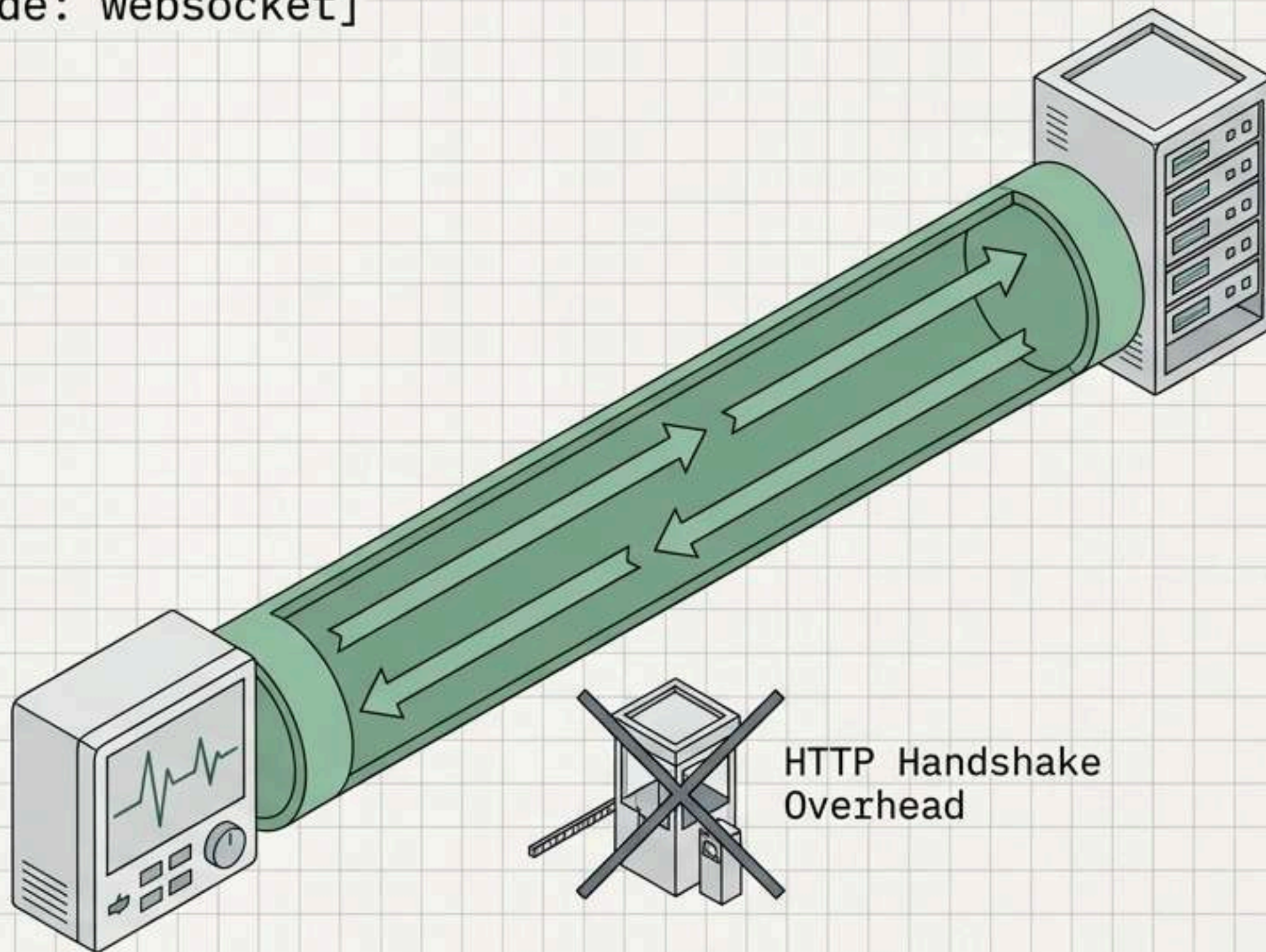
Trade-off Clave

↑ Ventaja

Comunicación casi instantánea en milisegundos.

↓ Costo

Alto consumo de infraestructura. Los servidores mantienen sockets abiertos en RAM. Escalar requiere buses inter-nodo (ej. Redis Pub/Sub). Riesgo de seguridad CSWSH.





WebHooks: Inversión de Control y Push Asíncrono

[HTTP POST] [Unidireccional] [Stateless]

Filosofía

El paradigma “No me llames, yo te llamo”. Una arquitectura dirigida por eventos donde la comunicación ocurre mediante peticiones HTTP POST salientes generadas activamente por el servidor hacia una URL pública del cliente.

Caso Práctico NeoMarket

La pasarela de pagos notifica asincrónicamente a `/api/webhooks/payments` cuando una transacción es aprobada, liberando al cliente de preguntar repetitivamente.

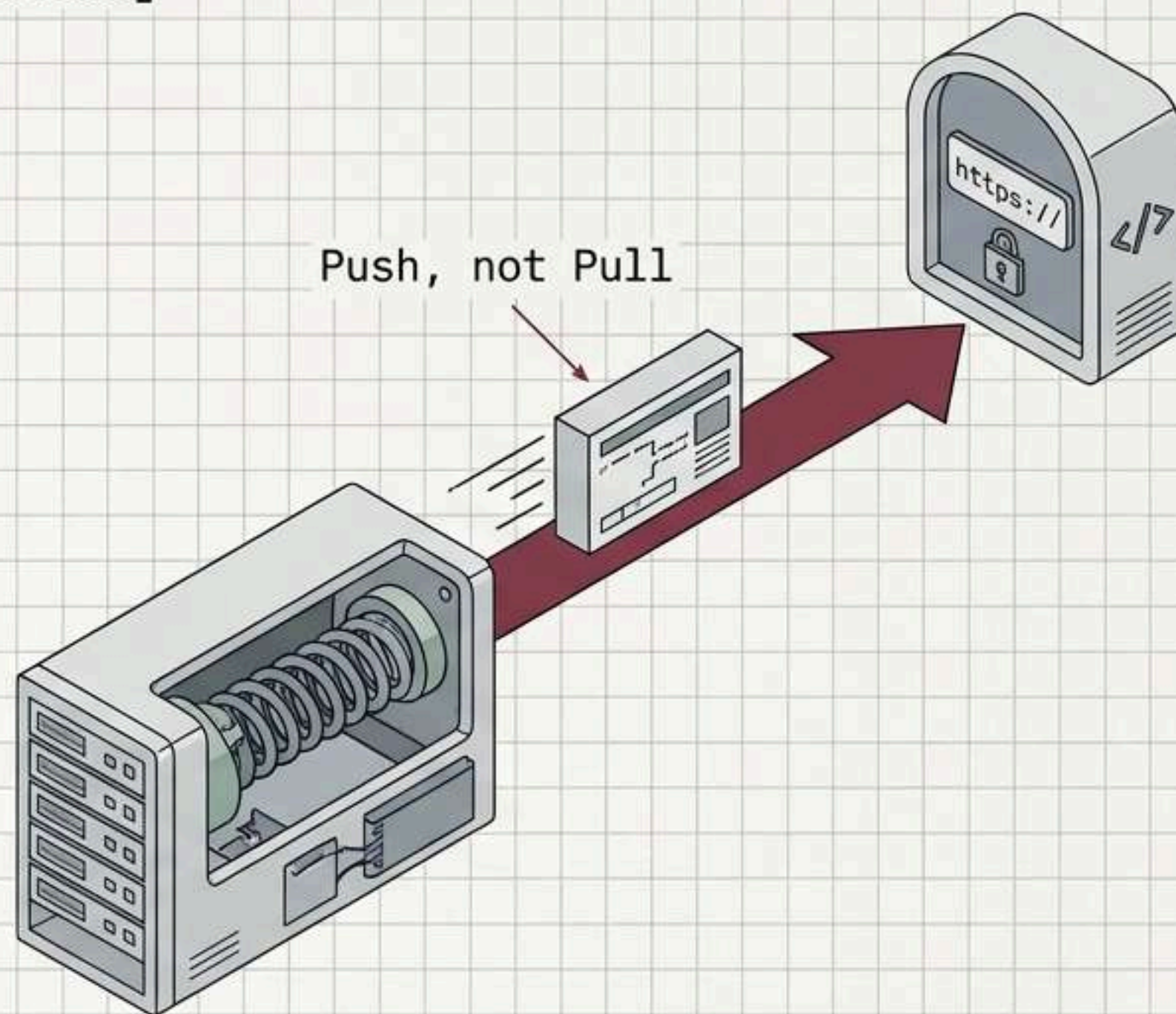
Trade-off Clave

↑ Ventaja

Altamente escalable, efímero y Stateless (libera recursos inmediatamente en el emisor).

↓ Costo

Seguridad perimetral crítica en el receptor. Obliga a implementar firmas criptográficas (ej. X-Hub-Signature) para validar la autenticidad del emisor.





Serverless / FaaS: La Frontera Elástica Efímera

[API Gateway] [Micro-funciones] [Efímero]

Filosofía

Lógica de negocio completamente fragmentada en micro-funciones atómicas. Su ejecución está estrechamente acoplada al ciclo de vida de un evento HTTP o disparador a través de un API Gateway.

Escalabilidad

Instantánea e infinita. El proveedor de nube asigna instancias bajo demanda por cada petición, reduciendo la superficie de ataque perimetral permanente.

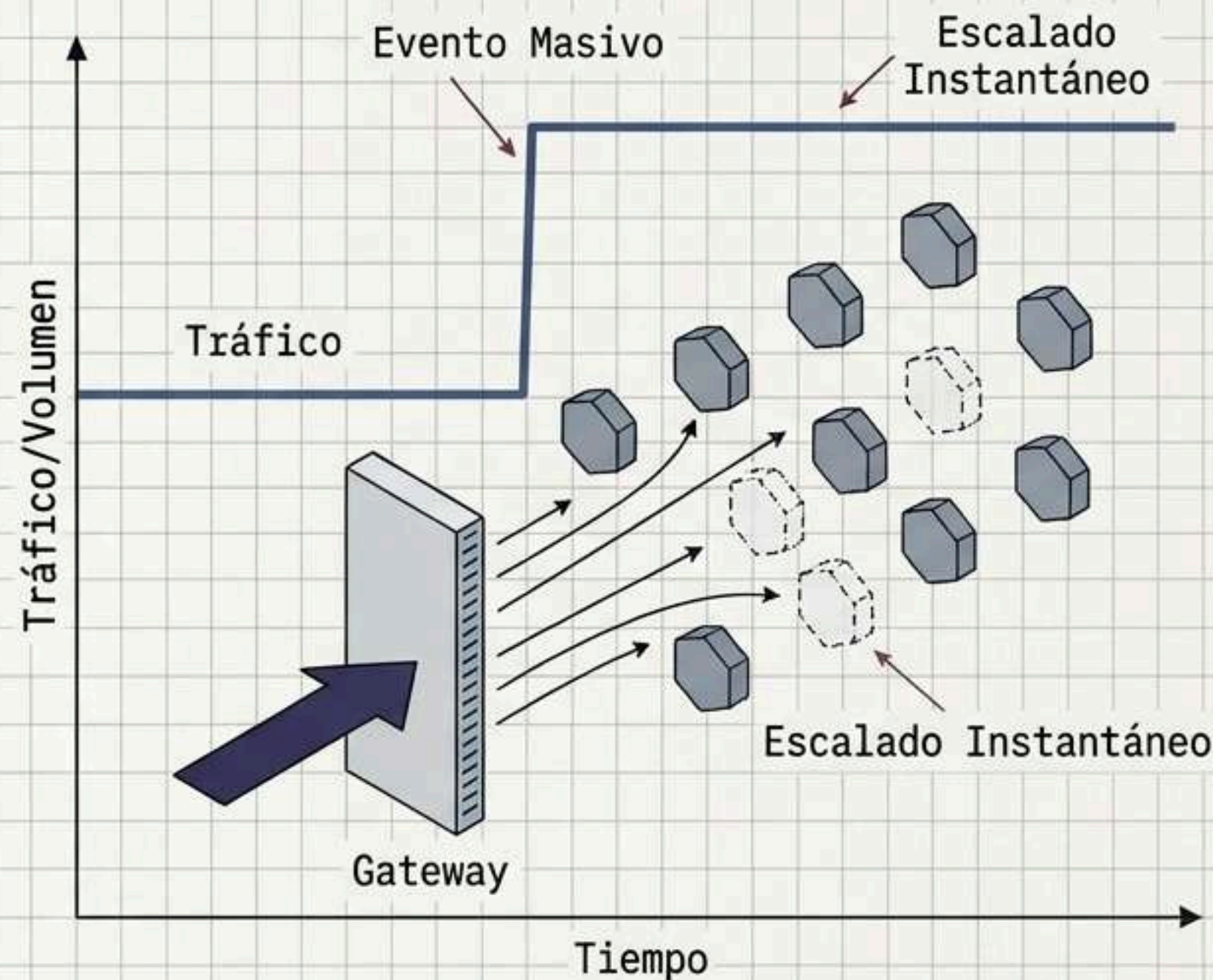
Trade-off Clave

↑ Ventaja

Cero gestión de infraestructura subyacente. Costo proporcional al uso exacto.

↓ Costo

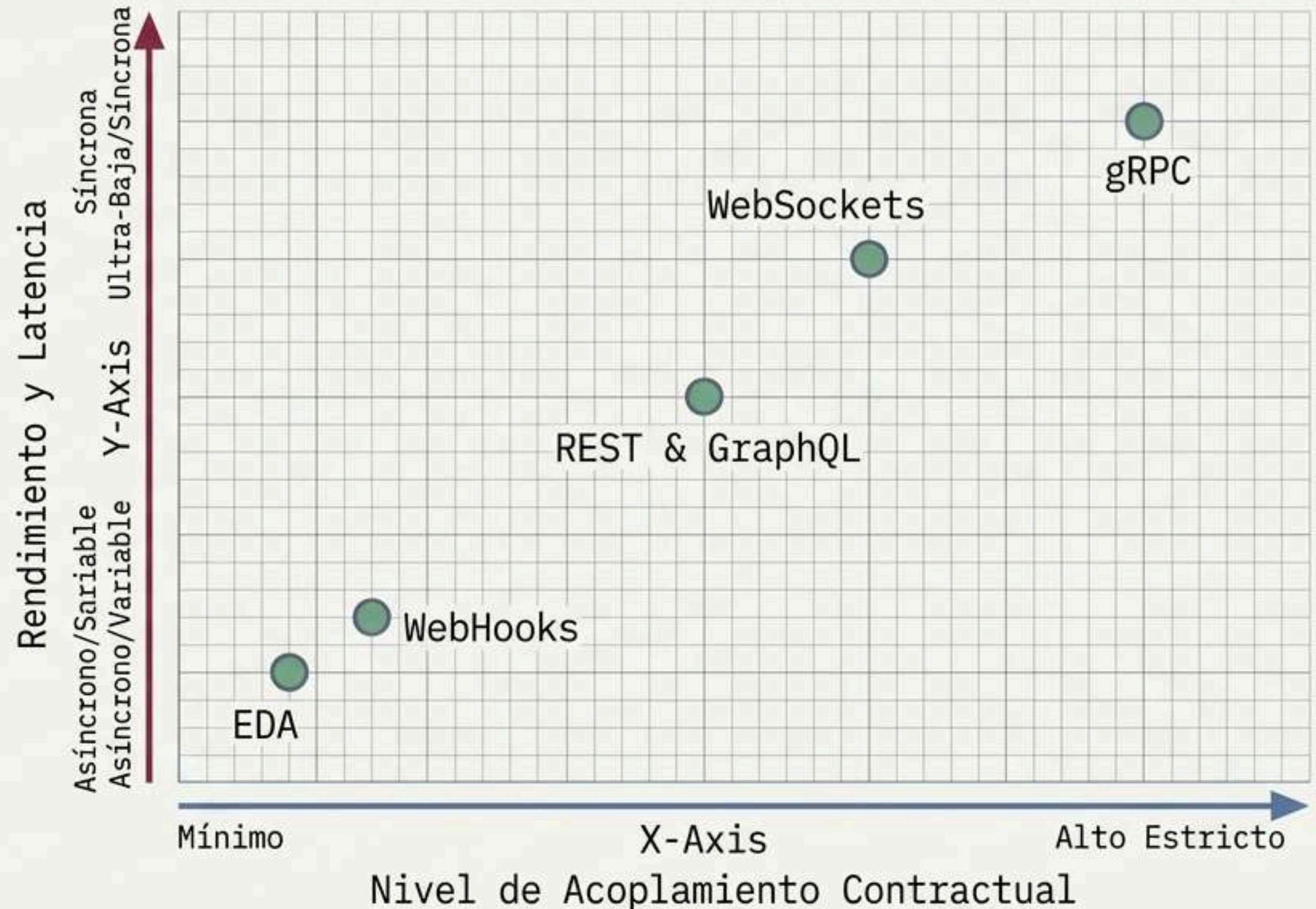
Sujeto al impacto de latencia por Cold Starts (arranques en frío) y un altísimo nivel de Vendor Lock-in tecnológico con el proveedor cloud.



Análisis de Fricción: Rendimiento vs. Acoplamiento

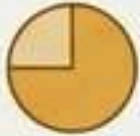
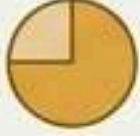
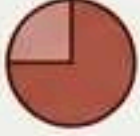
La física de los sistemas distribuidos revela una regla constante:

A mayor velocidad síncrona y optimización de red requerida (ej. gRPC), mayor compromiso contractual y rigidez exigimos de nuestros clientes y desarrolladores.



Evaluación ISO/IEC 25010:

Matriz de Atributos de Calidad

	REST	GraphQL	gRPC	EDA	FaaS	WebSockets	WebHooks
Facilidad de Cacheo				Dash/NA		Dash/NA	Dash/NA
Escalabilidad Concurrente							
Mantenibilidad de Código							
Seguridad Perimetral							

REST domina en resiliencia de lectura mediante cacheo de red nativo.

EDA triunfa en absorción de concurrencia, pero penaliza drásticamente la mantenibilidad de trazabilidad.

WebSockets ofrece latencia inigualable, pero rompe los patrones tradicionales de escalabilidad horizontal.

El Dilema del Tiempo Real



WebSockets: El Flujo Continuo

Direccionalidad: Full-Duplex (Paralelo).

Infraestructura: Stateful (Mantiene sockets persistentes en memoria RAM).

Escalabilidad: Compleja (No admite balanceadores simples; requiere Pub/Sub inter-nodo para coordinar).



WebHooks: El Evento Discreto

Direccionalidad: Unidireccional (Server-to-Client push).

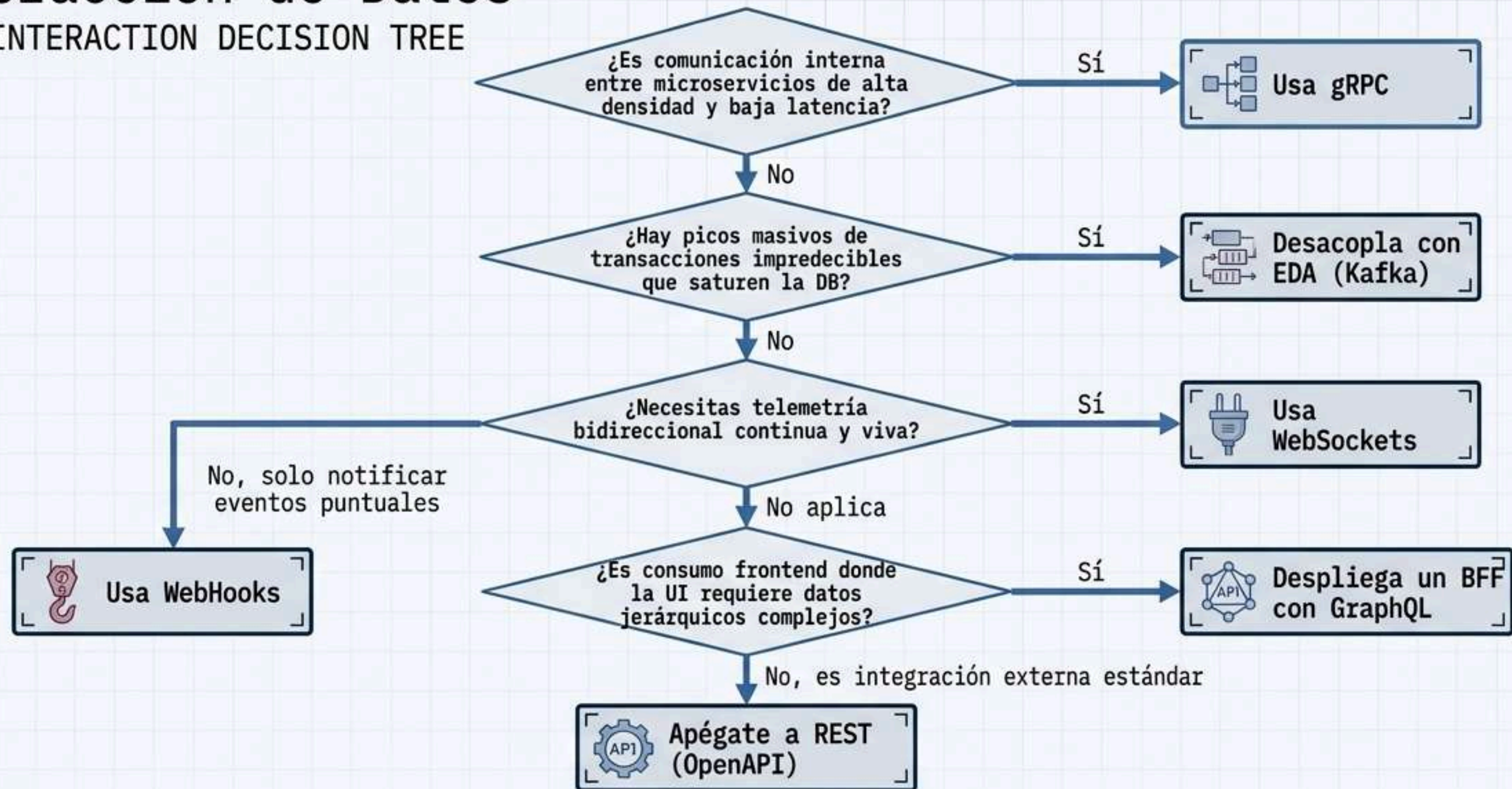
Infraestructura: Stateless (Efímero; ejecuta el POST, recibe HTTP 200 OK y libera recursos).

Escalabilidad: Alta (Sigue el estándar HTTP común con colas de reintentos y backoff).

El Veredicto del Tiempo Real: Utiliza **WebSockets** exclusivamente para flujos de datos continuos (Streams de IoT, Chat en vivo). Utiliza **WebHooks** para notificaciones asíncronas de eventos puntuales y discretos (Pagos, Cambios de estado).

Árbol de Decisiones de Interacción de Datos

DATA INTERACTION DECISION TREE



Principios Fundamentales del Diseño Distribuido

01

El Contrato Dicta la Velocidad

La forma en que produces y validas contratos (OpenAPI, Protobuf, AsyncAPI) determinará la agilidad de tus equipos más que la tecnología subyacente.

02

Abraza la Aislamiento Temporal

En sistemas a gran escala, asume la falla. Tecnologías asíncronas (EDA, WebHooks) previenen fallos en cascada al aislar en tiempo y espacio a los emisores de los receptores.

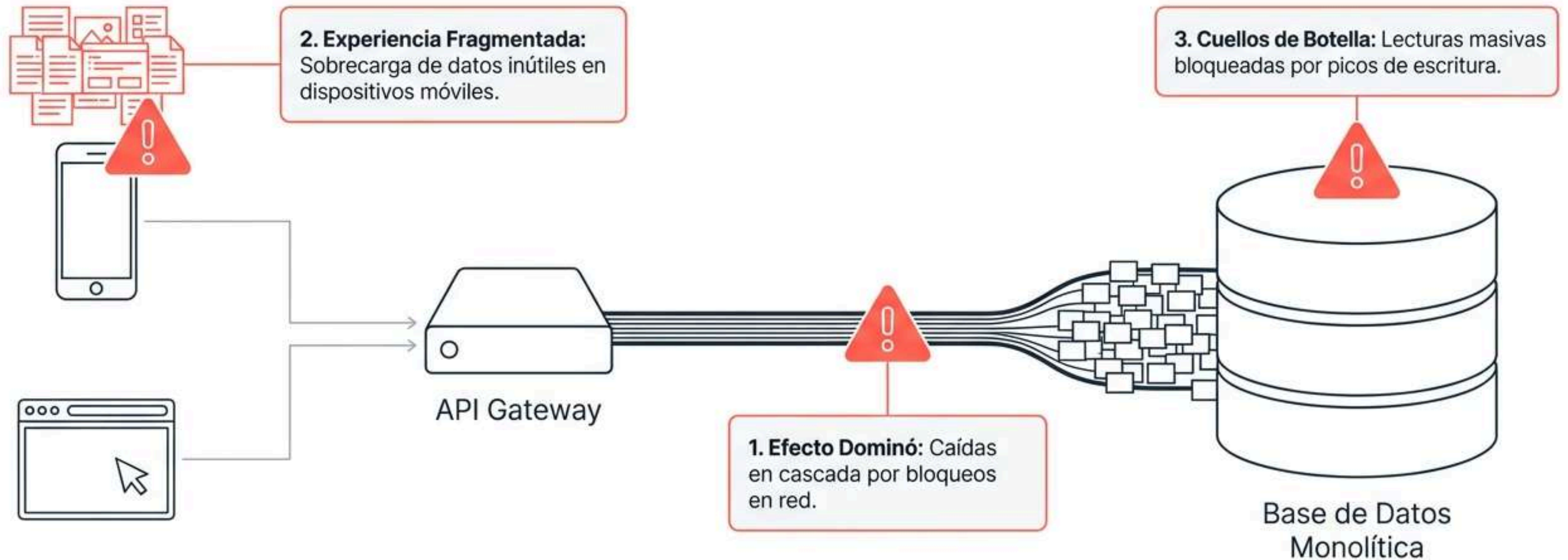
03

La Complejidad no se Destruye, se Transfiere

GraphQL facilita la experiencia del Frontend transfiriendo el peso al CPU del Backend. gRPC maximiza el rendimiento sacrificando la mantenibilidad del código.

“La arquitectura óptima es aquella donde los sacrificios realizados están alineados con el modelo de negocio.”

El Desafío NeoMarket: Anatomía de una Sobrecarga



El Marco de Solución



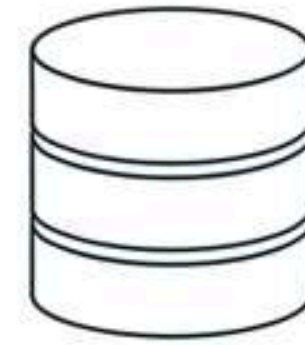
Resiliencia y Estabilidad

- Circuit Breaker
- Retry & Rate Limiting



Consumo y Optimización

- BFF (Backend For Frontend)
- API Composition



Persistencia y Estado

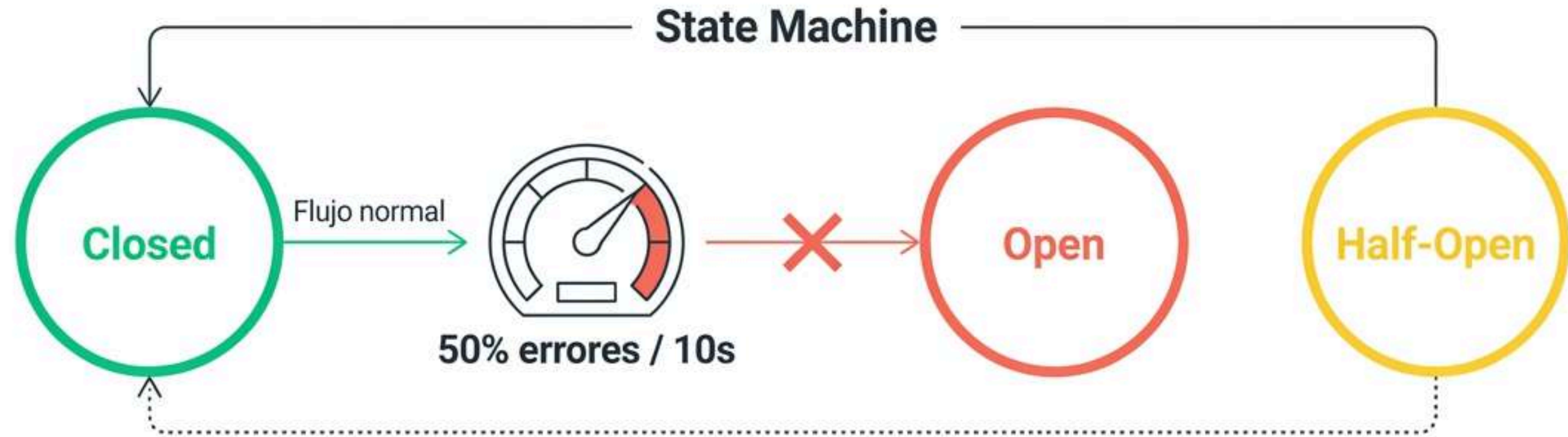
- CQRS
- Event Sourcing



Pilar 1: Resiliencia y Estabilidad

Protegiendo el API Gateway contra el agotamiento de hilos (thread starvation) y las fallas en cascada.

Circuit Breaker (El Disyuntor)



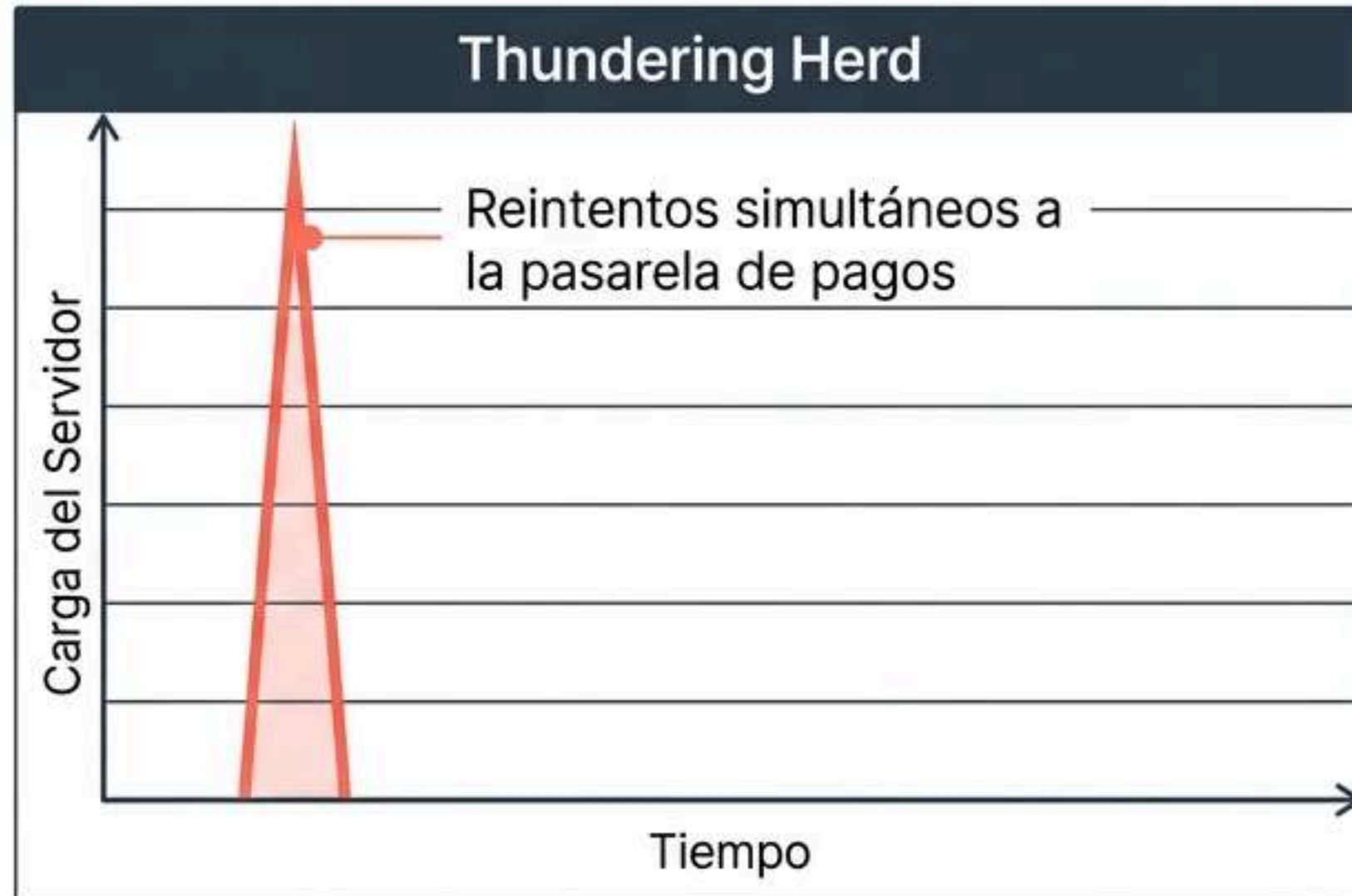
Síntoma (NeoMarket)

Validación de Partners tarda 10s, agotando el pool de hilos.

Mecánica

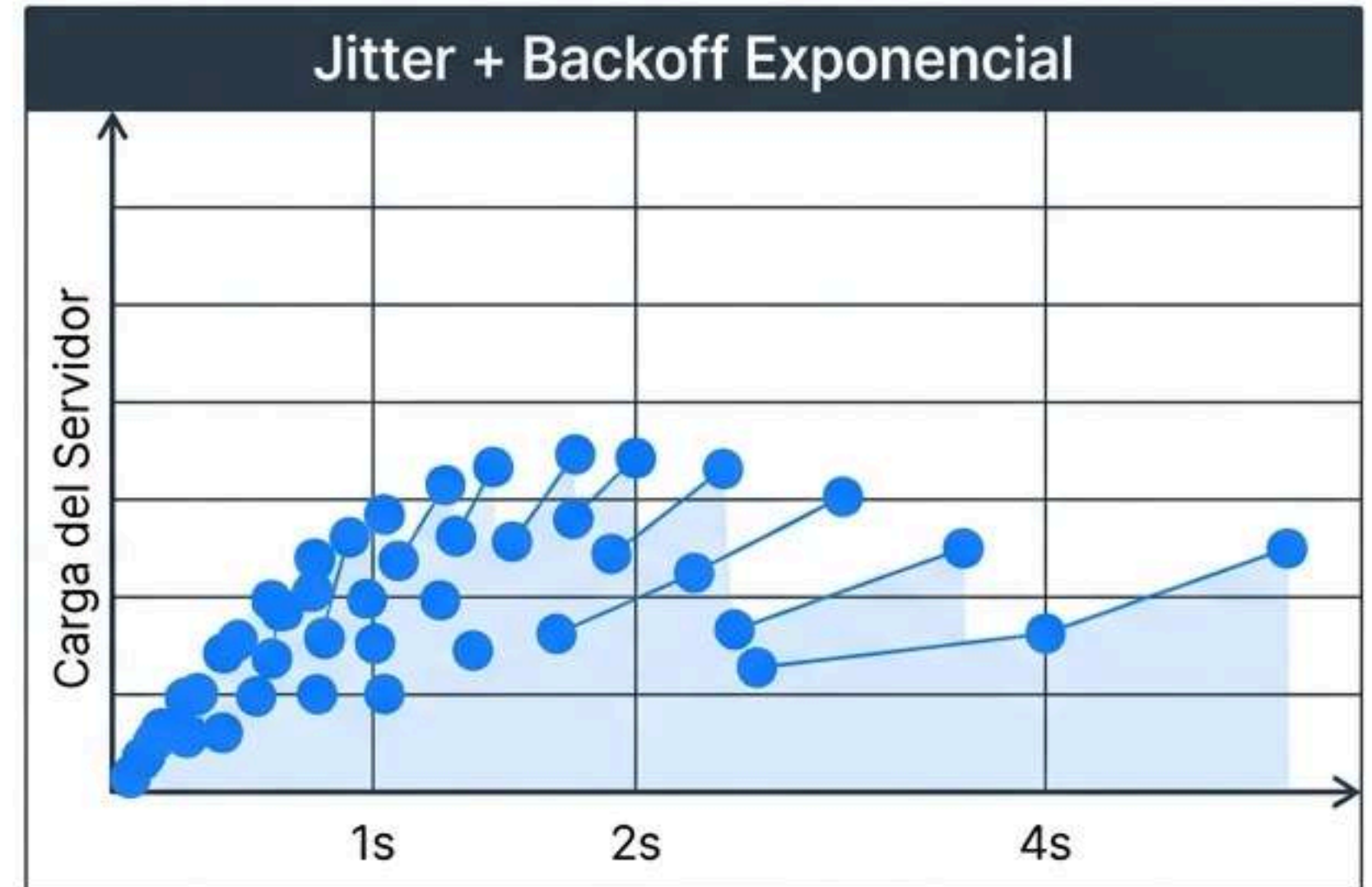
Intercepta fallos. Al superar el umbral, bloquea tráfico al instante devolviendo una respuesta degradada (caché) para salvar el API Gateway.

Retry: Evitando el Efecto Estampida



Síntoma (NeoMarket):

Fallos transitorios en pagos provocan reintentos masivos que **colapsan la red**.



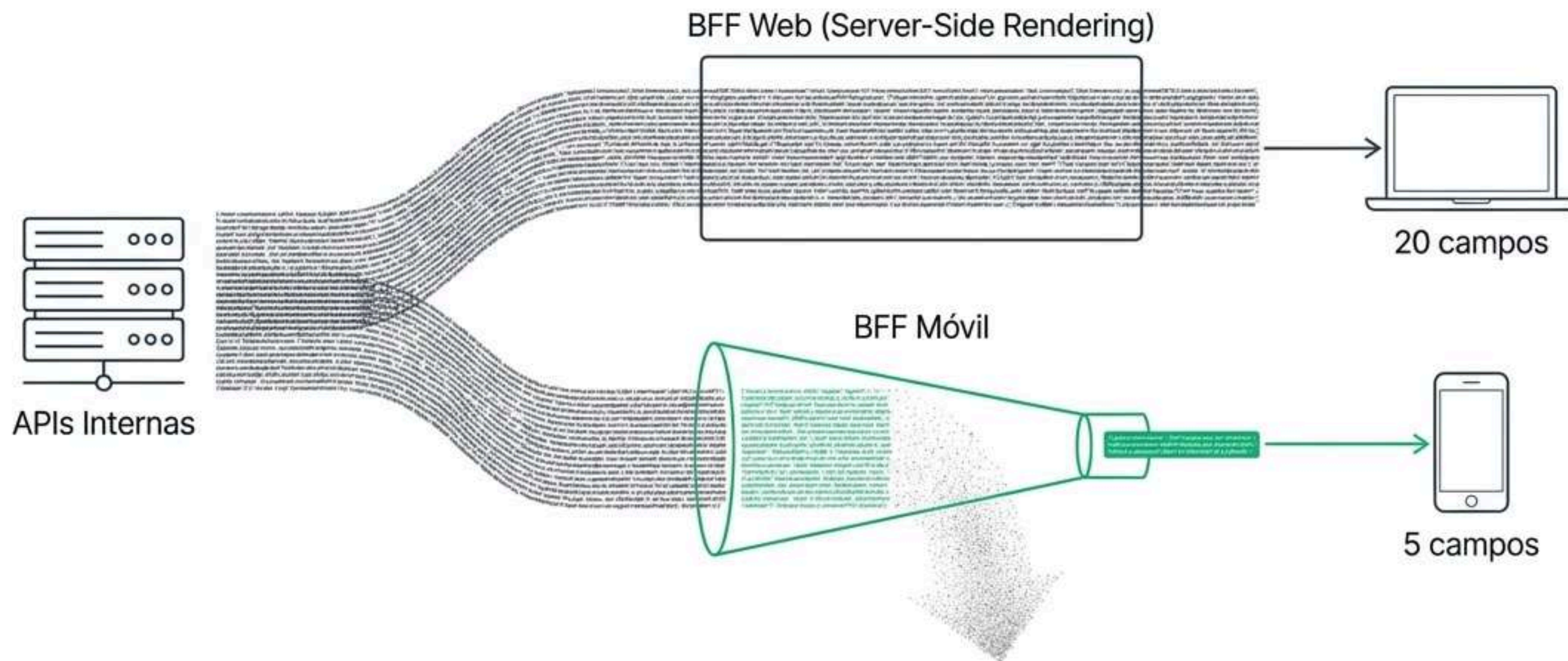
Mecánica: Retrasos automáticos crecientes (Backoff) más un factor aleatorio (Jitter) para distribuir la carga de recuperación.

A line-art illustration of a smartphone and a desktop monitor. A network diagram with nodes and connecting lines is overlaid on both devices. The text 'Pilar 2: Consumo y Optimización' is written in a large, bold, black font across the middle of the image, partially overlapping the devices.

Pilar 2: Consumo y Optimización

Desbloqueando el rendimiento del frontend y eliminando el over-fetching en entornos omnicanal.

Backend For Frontend (BFF)



Síntoma (NeoMarket):

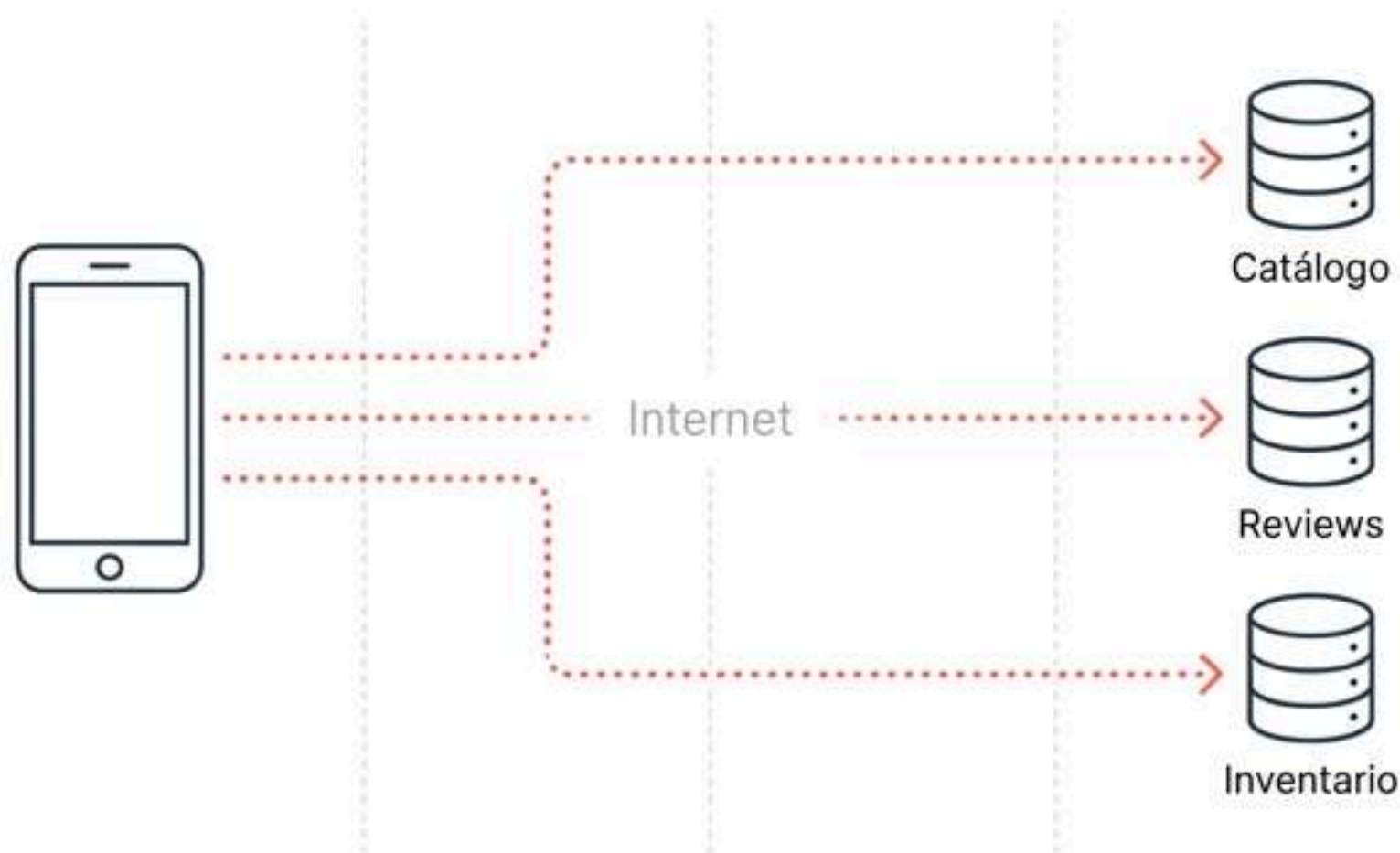
La App móvil descarga un JSON masivo del perfil de usuario y descarta el 75%.

Mecánica:

Capas de API exclusivas por cliente. El equipo móvil controla la limpieza, compresión y reducción de payloads.

API Composition

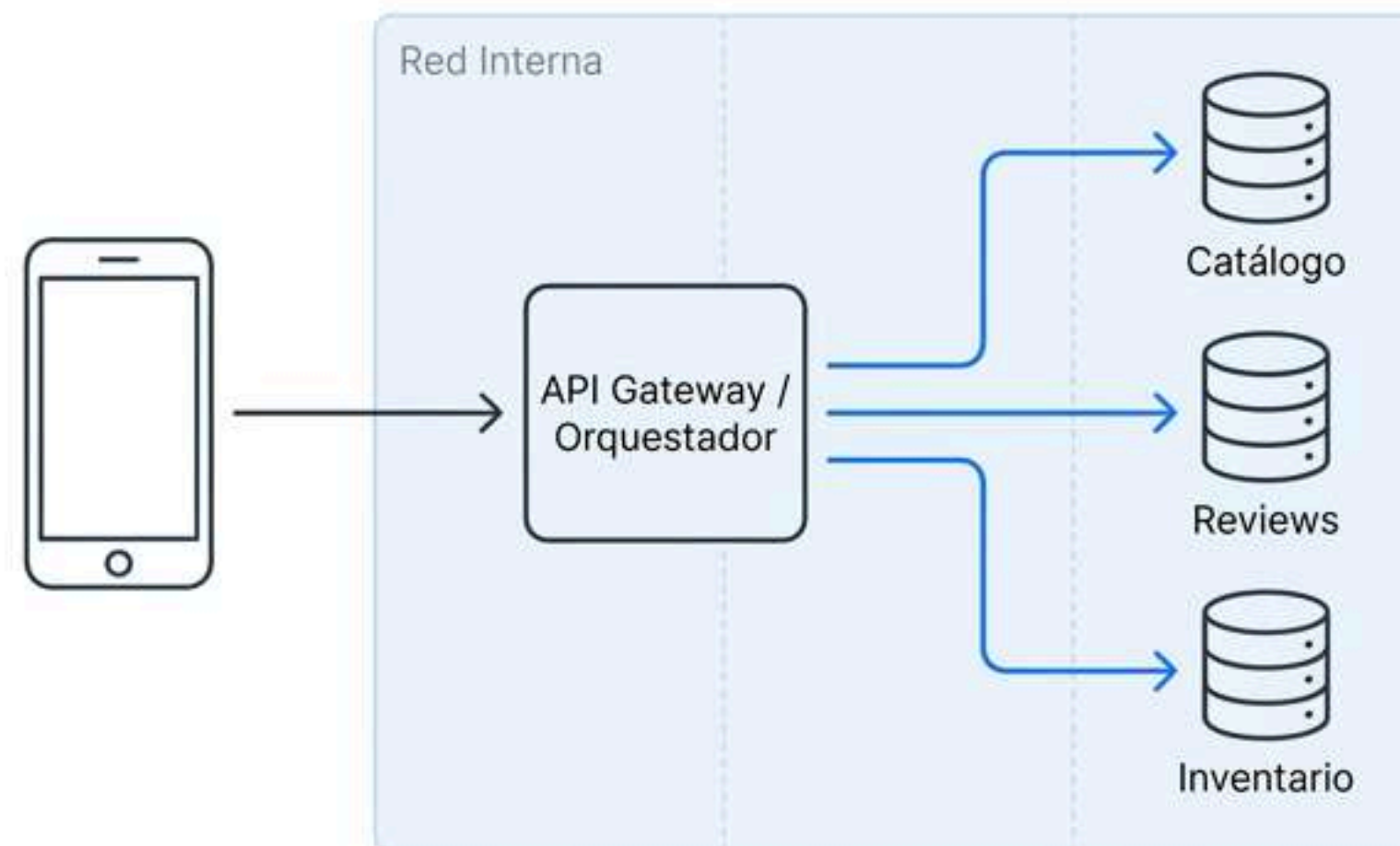
Antes - Lento



Síntoma (NeoMarket):

Cargar un producto requiere 3 llamadas HTTP desde el móvil, disparando la latencia.

Después - Optimizado



Mecánica:

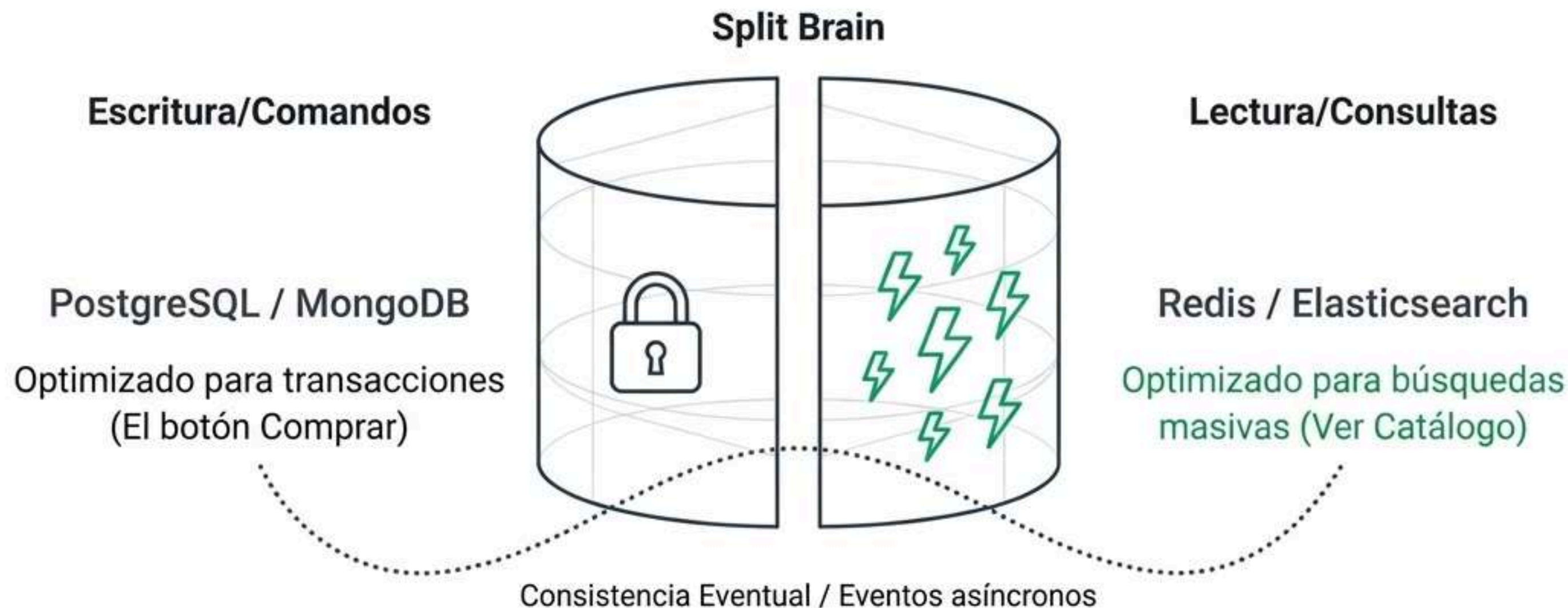
Una sola petición del cliente. El orquestador paraleliza internamente y une las respuestas (join) en un único JSON.



Pilar 3: Persistencia y Estado

Desacoplando lecturas de escrituras y garantizando una auditoría matemática perfecta bajo alta concurrencia.

CQRS (Separación de Responsabilidades)



Síntoma (NeoMarket):

Compras masivas bloquean las tablas de inventario durante Ofertas Flash.

Mecánica:

Separación física de modelos. Lecturas y escrituras escalan de manera completamente independiente.

Event Sourcing (El Historial Inmutable)



Síntoma (NeoMarket): Imposibilidad de auditar alteraciones erróneas en el saldo de comisiones de partners.

Mecánica: El estado no se sobrescribe; se calcula. Almacenamos la secuencia inmutable de eventos. El saldo final es una proyección matemática.

Matriz de Evolución de Datos

	Tradicional (CRUD)	Event-Driven (CQRS + ES)
Estado	Sobrescribe el estado anterior (Pérdida de historia).	Añade eventos inmutables al log (Append-only).
Rendimiento	Lecturas y escrituras compiten por recursos.	Lecturas desnormalizadas asíncronas ultra-rápidas.
Auditoría	Requiere tablas de log separadas e imperfectas.	Auditoría matemática perfecta por diseño (Fuente de verdad).

Matriz de Diagnóstico de Patrones API

Síntoma Observado	Patrón Recomendado	Resultado Esperado
Cascadas de error por validaciones lentas	Circuit Breaker	Fallo rápido, protege pool de hilos.
Caídas por picos de reintentos simultáneos	Retry + Jitter	Carga distribuida en el tiempo.
Dispositivos móviles descartando grandes payloads	BFF	Payloads quirúrgicos por cliente.
Alta latencia por múltiples llamadas HTTP	API Composition	Paralelización en red interna.
Consultas lentas por bloqueos transaccionales	CQRS	Escalabilidad de lectura independiente.